```
            <button class="step-btn" type="button" @click="stepCount(1)">+1</button>
            <button class="step-btn" type="button" @click="stepCount(10)">+10</button>
          </div>
          <div class="quick-counts result-counts" aria-label="常用数字">
            <button
              v-for="num in QUICK_COUNTS"
              :key="`result-${num}`"
              class="quick-chip"
              type="button"
              :aria-pressed="currentCount === num"
              @click="jumpToCount(num)"
            >
              {{ num }}
            </button>
          </div>
          <button class="play-again-btn" @click="resetExplore">
            <RotateCcw :size="20" aria-hidden="true" />
            <span>重新输入</span>
          </button>
        </section>
      </main>
    </Transition>
  </div>
</template>
<style scoped>
.explore-page {
  min-height: 100dvh;
  display: flex;
  flex-direction: column;
  padding-top: env(safe-area-inset-top);
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);
  background:
    radial-gradient(circle at top, rgba(92, 157, 255, 0.12) 0%, rgba(92, 157, 255, 0) 38%),
    linear-gradient(180deg, #f8fbff 0%, #edf4ff 100%);
  overflow: hidden;
}
.top-bar {
  display: flex;
  align-items: center;
  padding: 12px 16px 4px;
}
.back-btn {
  width: 48px;
  height: 48px;
  border: 1px solid rgba(92, 157, 255, 0.12);
  border-radius: 18px;
  background: rgba(255, 255, 255, 0.88);
  box-shadow: 0 12px 28px rgba(49, 120, 246, 0.12);
  display: inline-flex;
```

```
align-items: center;
```

```
  justify-content: center;
  color: var(--brand-primary);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), box-shadow var(--
duration-fast) var(--ease-standard);
}
.back-btn:focus-visible,
.play-again-btn:focus-visible,
.quick-chip:focus-visible,
.step-btn:focus-visible,
.mode-btn:focus-visible,
.secondary-btn:focus-visible,
.range-chip:focus-visible {
  outline: none;
  box-shadow: 0 0 0 4px rgba(92, 157, 255, 0.18);
}
.back-btn:active,
.play-again-btn:active {
  transform: scale(0.95);
}
.view-wrapper {
  flex: 1;
  display: flex;
  flex-direction: column;
}
.input-screen,
.result-screen {
  flex: 1;
  min-height: 0;
  display: flex;
  flex-direction: column;
  padding: 8px 14px 0;
}
.input-screen {
  gap: 12px;
}
.result-screen {
  gap: 12px;
  padding-inline: 10px;
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);
  overflow-y: auto;
  -webkit-overflow-scrolling: touch;
}
.mode-switch {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
}
```

```
.mode-btn,
.secondary-btn {
```

```
  min-height: 48px;
  border: 1px solid rgba(92, 157, 255, 0.14);
  border-radius: 18px;
  background: rgba(255, 255, 255, 0.84);
  color: var(--text-blue);
  font-size: 15px;
  font-weight: 800;
  transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), border-color var(--
duration-fast) var(--ease-standard);
}
.mode-btn.is-active {
  background: linear-gradient(180deg, #6da8ff 0%, #4b86f3 100%);
  border-color: rgba(75, 134, 243, 0.24);
  color: #fff;
  box-shadow: 0 12px 22px rgba(75, 134, 243, 0.18);
}
.range-selector-shell,
.challenge-workspace {
  border: 1px solid rgba(92, 157, 255, 0.12);
  border-radius: 30px;
  background: rgba(255, 255, 255, 0.9);
  box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);
  backdrop-filter: blur(14px);
  -webkit-backdrop-filter: blur(14px);
}
.range-selector-shell {
  padding: 10px 12px;
}
.range-selector-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 8px;
}
.range-title-badge {
  flex-shrink: 0;
}
.range-selector-copy {
  margin: 0;
  color: var(--text-blue-light);
  font-size: 12px;
  font-weight: 700;
  text-align: right;
}
.challenge-workspace {
  padding: 8px;
  display: flex;
  flex-direction: column;
```

```
  gap: 8px;
}
```

```
.explore-workspace {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
.number-stage,
.result-number-shell,
.play-again-btn,
.step-btn,
.challenge-preview-shell {
  position: relative;
  border: 1px solid rgba(92, 157, 255, 0.12);
  background: rgba(255, 255, 255, 0.88);
  box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);
  backdrop-filter: blur(14px);
  -webkit-backdrop-filter: blur(14px);
}
.number-stage,
.result-number-shell,
.challenge-preview-shell {
  overflow: hidden;
  border-radius: 34px;
}
.number-stage {
  padding: 10px 12px 12px;
  background: rgba(255, 255, 255, 0.96);
  border: 1px solid rgba(92, 157, 255, 0.14);
  box-shadow: 0 20px 36px rgba(58, 87, 152, 0.1);
}
.number-card {
  width: 100%;
}
.number-card-top,
.number-display {
  display: flex;
  align-items: center;
}
.number-card-top {
  justify-content: flex-start;
  gap: 8px;
  margin-bottom: 8px;
}
.counter-badge {
  padding: 8px 12px;
  border-radius: var(--radius-full);
  background: rgba(92, 157, 255, 0.1);
  border: 1px solid rgba(92, 157, 255, 0.12);
  color: var(--brand-primary);
  font-size: var(--font-sm);
```

```
font-weight: 800;
```

```
}
.range-badge {
  background: rgba(107, 203, 119, 0.12);
  border-color: rgba(107, 203, 119, 0.18);
  color: var(--text-teal);
}
.number-display {
  justify-content: center;
  min-height: 76px;
}
.big-number {
  position: relative;
  z-index: 1;
  min-width: clamp(88px, 24vw, 132px);
  padding: 12px 14px;
  border-radius: var(--radius-lg);
  border: 2px solid rgba(92, 157, 255, 0.18);
  background: linear-gradient(180deg, #ffffff 0%, #f1f7ff 100%);
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.92), 0 10px 20px rgba(92, 157, 255, 0.08);
  color: var(--text-blue-dark);
  font-size: clamp(48px, 14vw, 76px);
  font-weight: 800;
  line-height: 1;
  text-align: center;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), box-shadow var(-
-duration-fast) var(--ease-standard);
}
.big-number.is-empty {
  color: var(--text-muted);
}
.status-copy,
.result-copy {
  margin: 0;
  text-align: center;
}
.status-copy {
  min-height: 16px;
  margin-top: 6px;
  color: var(--text-blue-light);
  font-size: 12px;
  font-weight: 700;
  line-height: 1.25;
}
.status-copy.is-warning {
  color: var(--text-warning);
}
.status-copy.is-success {
  color: var(--text-success);
}
```

```
.challenge-meta,
.challenge-summary {
```

```
  display: grid;
  gap: 6px;
  margin-top: 8px;
}
.challenge-meta {
  grid-template-columns: repeat(2, minmax(0, 1fr));
}
.challenge-summary {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
.range-selector {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 6px;
  margin-top: 2px;
}
.range-chip {
  min-height: 44px;
  padding: 0 12px;
  border: 1px solid rgba(92, 157, 255, 0.16);
  border-radius: 999px;
  background: rgba(255, 255, 255, 0.9);
  color: var(--text-blue);
  font-size: 13px;
  font-weight: 800;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--
duration-fast) var(--ease-standard), color var(--duration-fast) var(--ease-standard);
}
.range-chip.is-active {
  background: linear-gradient(180deg, #6ecb96 0%, #4aac70 100%);
  border-color: rgba(74, 172, 112, 0.22);
  color: #fff;
  box-shadow: 0 10px 18px rgba(74, 172, 112, 0.16);
}
.challenge-card {
  min-height: 44px;
  padding: 6px 4px;
  border: 1px solid rgba(92, 157, 255, 0.12);
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.82);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 2px;
  text-align: center;
}
.challenge-label {
```

```
  color: var(--text-blue-light);
  font-size: 10px;
```

```
  font-weight: 700;
}
.challenge-card strong {
  color: var(--text-blue-dark);
  font-size: 13px;
  font-weight: 900;
}
.quick-counts {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 8px;
  margin-top: 14px;
}
.quick-chip {
  min-height: 44px;
  padding: 0 14px;
  border: 1px solid rgba(92, 157, 255, 0.16);
  border-radius: 999px;
  background: rgba(255, 255, 255, 0.9);
  color: var(--brand-primary);
  font-size: 14px;
  font-weight: 800;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--
duration-fast) var(--ease-standard);
}
.quick-chip:active,
.step-btn:active,
.mode-btn:active,
.secondary-btn:active,
.range-chip:active {
  transform: scale(0.96);
}
.number-stage.is-shaking .big-number {
  border-color: rgba(255, 107, 107, 0.4);
  animation: toy-shake 0.28s ease;
}
.number-stage.is-shaking {
  animation: none;
}
.challenge-preview-shell {
  padding: 8px;
  border-radius: 22px;
  background: linear-gradient(180deg, rgba(247, 250, 255, 0.98) 0%, rgba(235, 243, 255, 0.95) 100%);
}
.preview-copy {
  display: inline-flex;
  align-items: center;
  gap: 8px;
```

```
color: var(--text-blue);
font-size: 12px;
```

```
  font-weight: 800;
  margin-bottom: 6px;
}
.challenge-ball-shell {
  border-radius: 20px;
  overflow: hidden;
}
.result-number-shell {
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 18px 18px 16px;
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(241, 247, 255, 0.96) 100%);
}
.result-number-block {
  width: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 10px;
}
.result-badge {
  background: rgba(107, 203, 119, 0.12);
  border-color: rgba(107, 203, 119, 0.18);
  color: var(--text-teal);
}
.result-copy {
  color: var(--text-blue-light);
  font-size: 15px;
  font-weight: 800;
  line-height: 1.5;
}
.result-ball-shell {
  width: 100%;
  max-width: min(100%, calc(100dvh - 320px));
  min-height: 236px;
  margin: 0 auto;
  border-radius: 34px;
  overflow: hidden;
}
.result-ball-shell--wide {
  aspect-ratio: 16 / 9;
}
.result-ball-shell--landscape {
  aspect-ratio: 4 / 3;
}
.result-ball-shell--square {
  aspect-ratio: 1 / 1;
}
```

```
.result-tools {
```

```
  display: flex;
  flex-direction: column;
  gap: 12px;
}
.step-controls {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
}
.step-btn {
  min-height: 52px;
  border-radius: 20px;
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(242, 247, 255, 0.96) 100%);
  color: var(--text-blue-dark);
  font-size: 18px;
  font-weight: 900;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--
duration-fast) var(--ease-standard);
}
.result-counts {
  margin-top: 0;
}
.play-again-btn {
  width: 100%;
  min-height: 64px;
  border-radius: 24px;
  padding: 0 20px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  color: #fff;
  font-size: 22px;
  font-weight: 900;
  background: linear-gradient(180deg, #62c98a 0%, #48aa70 100%);
  border-color: rgba(74, 172, 112, 0.24);
  box-shadow: 0 12px 26px rgba(74, 172, 112, 0.2);
}
.secondary-btn {
  color: var(--text-blue-light);
}
@media (hover: hover) {
  .quick-chip:hover,
  .step-btn:hover,
  .secondary-btn:hover,
  .range-chip:hover:not(.is-active) {
    border-color: rgba(92, 157, 255, 0.28);
    background: var(--bg-light);
  }
```

```
.mode-btn:hover:not(.is-active) {
  background: rgba(255, 255, 255, 0.94);
```

```
  }
  .play-again-btn:hover {
    box-shadow: 0 16px 30px rgba(74, 172, 112, 0.24);
  }
}
.fade-enter-active,
.fade-leave-active {
  transition: opacity 0.35s ease, transform 0.35s ease;
}
.fade-enter-from,
.fade-leave-to {
  opacity: 0;
  transform: scale(0.96) translateY(20px);
}
@media (max-width: 420px) {
  .range-selector-header {
    flex-direction: column;
    align-items: flex-start;
  }
  .range-selector-copy {
    text-align: left;
  }
  .number-stage {
    padding: 8px 10px 10px;
  }
  .number-card-top {
    margin-bottom: 8px;
  }
  .number-display {
    min-height: 66px;
  }
  .big-number {
    min-width: 72px;
    padding: 8px 10px;
  }
  .challenge-meta {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
  .challenge-summary {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
  .challenge-workspace {
    padding: 6px;
    gap: 6px;
  }
  .result-copy {
    font-size: 14px;
  }
  .result-ball-shell {
```

```
min-height: 212px;
```

```
  }
  .step-controls {
    gap: 8px;
  }
  .step-btn {
    min-height: 48px;
    border-radius: 18px;
    font-size: 17px;
  }
}
@media (max-width: 360px) {
  .counter-badge {
    padding: 8px 10px;
    font-size: 12px;
  }
  .number-display {
    min-height: 92px;
  }
  .big-number {
    font-size: 42px;
  }
}
@media (max-width: 959px) and (max-height: 860px) {
  .number-stage {
    padding-top: 8px;
    padding-bottom: 8px;
  }
  .number-display {
    min-height: 88px;
  }
  .result-ball-shell {
    max-width: min(100%, calc(100dvh - 300px));
    min-height: 196px;
  }
}
@media (min-width: 768px) {
  .number-stage {
    padding: 14px 18px 18px;
  }
}
@media (min-width: 960px) {
  .result-screen {
    width: min(100%, 1120px);
    margin: 0 auto;
    padding-inline: 24px;
    display: grid;
    grid-template-columns: minmax(280px, 360px) minmax(420px, 680px);
    grid-template-rows: auto 1fr;
    align-items: start;
```

```
    justify-content: center;
```

```
    gap: 18px;
  }
  .result-number-shell {
    min-height: 220px;
  }
  .result-ball-shell {
    grid-column: 2;
    grid-row: 1 / span 2;
    max-width: min(100%, 680px);
    min-height: 420px;
    align-self: stretch;
  }
  .result-ball-shell--wide,
  .result-ball-shell--landscape {
    min-height: 0;
    align-self: start;
  }
  .result-tools {
    grid-column: 1;
  }
  .result-counts {
    justify-content: flex-start;
  }
}
</style>
// File: src/components/BallArray.vue
<script setup>
import { computed, nextTick, onMounted, onUnmounted, ref, watch } from 'vue'
import { Move3D, RotateCcw } from 'lucide-vue-next'
const props = defineProps({
  count: {
    type: Number,
    required: true,
    validator: (v) => v >= 1 && v <= 1000
  },
  size: {
    type: String,
    default: 'normal',
    validator: (v) => ['normal', 'compact'].includes(v)
  }
})
const canvasRef = ref(null)
const isLoading = ref(true)
const touchDevice = ref(false)
const isRotationEnabled = ref(false)
const renderError = ref(false)
const isSceneReady = ref(false)
const prefersReducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches
const showRotationControls = computed(() => (
```

```
touchDevice.value && isSceneReady.value && props.size === 'normal' && props.count >= 100
```

```
))
const ariaLabel = computed(() => {
  const hundreds = Math.floor(props.count / 100)
  const tens = Math.floor((props.count % 100) / 10)
  const ones = props.count % 10
  const parts = []
  if (hundreds) parts.push(`${hundreds} 个百`)
  if (tens) parts.push(`${tens} 个十`)
  if (ones || !parts.length) parts.push(`${ones} 个一`)
  return `${props.count} 颗小球，按十进制排列为${parts.join('、')}`
})
let scene = null
let camera = null
let renderer = null
let controls = null
let instancedMesh = null
let animationId = null
let renderId = null
let introStartTime = 0
let resizeObserver = null
let orbitDomElement = null
let controlsChangeHandler = null
let THREE = null
let animationDummy = null
function isTouchDevice() {
  return window.matchMedia('(pointer: coarse)').matches || 'ontouchstart' in window
}
const colors = {
  ballLow: 0x8FD0FF,
  ballMid: 0x78AEFF,
  ballHigh: 0x5B8FF2,
  ballPeak: 0x527FE2
}
function getBallPalette(count) {
  if (count <= 19) {
    return {
      color: colors.ballLow
    }
  }
  if (count <= 99) {
    return {
      color: colors.ballMid
    }
  }
  if (count <= 300) {
    return {
      color: colors.ballHigh
    }
  }
```

```
return {
```

```
    color: colors.ballPeak
  }
}
function getGroupedBallColor(count, index) {
  const palette = getBallPalette(count)
  const color = new THREE.Color(palette.color)
  if (count < 20) {
    const lightnessBoost = (index % 5) * 0.018
    color.offsetHSL(0, 0, lightnessBoost)
    return color
  }
  if (count < 100) {
    const tenGroup = Math.floor(index / 10)
    const hueShift = (tenGroup % 2 === 0 ? -1 : 1) * 0.012
    const lightnessShift = (tenGroup % 2 === 0 ? 1 : -1) * 0.025
    color.offsetHSL(hueShift, 0.015, lightnessShift)
    return color
  }
  const hundredGroup = Math.floor(index / 100)
  const tenGroup = Math.floor((index % 100) / 10)
  const alternateHundred = hundredGroup % 2 === 1
  const hueShift = alternateHundred ? 0.022 : -0.012
  const saturationShift = alternateHundred ? -0.012 : 0.012
  const hundredLightnessShift = alternateHundred ? 0.045 : -0.018
  const tenLightnessShift = tenGroup % 2 === 0 ? 0.008 : -0.008
  const lightnessShift = hundredLightnessShift + tenLightnessShift
  color.offsetHSL(hueShift, saturationShift, lightnessShift)
  return color
}
async function loadThree() {
  if (THREE) return
  const threeModule = await import('three')
  THREE = threeModule
}
function cleanupControls() {
  if (orbitDomElement) {
    orbitDomElement.style.touchAction = ''
  }
  if (controls && controlsChangeHandler) {
    controls.removeEventListener('change', controlsChangeHandler)
  }
  controlsChangeHandler = null
  orbitDomElement = null
  if (controls) {
    controls.dispose()
    controls = null
  }
}
function updateControlsInteraction() {
```

```
if (!controls || !orbitDomElement || !THREE) return
```

```
  if (!touchDevice.value) {
    controls.enabled = true
    controls.enableRotate = true
    controls.enableZoom = true
    controls.enablePan = true
    controls.touches = {
      ONE: THREE.TOUCH.ROTATE,
      TWO: THREE.TOUCH.DOLLY
    }
    orbitDomElement.style.touchAction = 'none'
    return
  }
  const enabled = showRotationControls.value && isRotationEnabled.value
  controls.enabled = enabled
  controls.enableRotate = enabled
  controls.enableZoom = false
  controls.enablePan = false
  controls.touches = {
    ONE: THREE.TOUCH.ROTATE,
    TWO: THREE.TOUCH.NONE
  }
  orbitDomElement.style.touchAction = enabled ? 'none' : 'pan-y'
}
async function loadOrbitControls() {
  if (controls) return
  await loadThree()
  const { OrbitControls } = await import('three/addons/controls/OrbitControls.js')
  orbitDomElement = renderer.domElement
  controls = new OrbitControls(camera, orbitDomElement)
  controls.enableDamping = true
  controls.dampingFactor = 0.08
  controls.minDistance = 3
  controls.maxDistance = 25
  controls.autoRotate = false
  controlsChangeHandler = () => { scheduleRender() }
  controls.addEventListener('change', controlsChangeHandler)
  controls.autoRotateSpeed = 0.7
  controls.rotateSpeed = 0.6
  controls.zoomSpeed = 0.8
  updateControlsInteraction()
}
function clearContainer(container) {
  while (container.firstChild) {
    container.removeChild(container.firstChild)
  }
}
async function initScene() {
  if (!canvasRef.value) return
  renderError.value = false
```

```
isSceneReady.value = false
```

```
try {
  await loadThree()
  await nextTick()
  const container = canvasRef.value
  if (!container) return
  const width = container.clientWidth || 320
  const height = container.clientHeight || 320
  cleanup()
  cleanupControls()
  scene = new THREE.Scene()
  scene.background = null
  camera = new THREE.PerspectiveCamera(32, width / height, 0.1, 200)
  renderer = new THREE.WebGLRenderer({
    antialias: true,
    alpha: true,
    powerPreference: 'high-performance'
  })
  renderer.setSize(width, height)
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
  renderer.shadowMap.enabled = false
  renderer.outputColorSpace = THREE.SRGBColorSpace
  renderer.toneMapping = THREE.NoToneMapping
  while (container.firstChild) {
    container.removeChild(container.firstChild)
  }
  container.appendChild(renderer.domElement)
  const ambientLight = new THREE.HemisphereLight(0xF8FBFF, 0xDCEBFF, 1.22)
  scene.add(ambientLight)
  const keyLight = new THREE.DirectionalLight(0xFFFFFF, 1.06)
  keyLight.position.set(6, 8, 11)
  scene.add(keyLight)
  const rimLight = new THREE.DirectionalLight(0xA8D7FF, 0.62)
  rimLight.position.set(-7, 5, 8)
  scene.add(rimLight)
  const bounceLight = new THREE.PointLight(0xCBE6FF, 0.6, 32, 2)
  bounceLight.position.set(0, -4, 7)
  scene.add(bounceLight)
  await loadOrbitControls()
  isSceneReady.value = true
  await nextTick()
  syncRendererSize()
  updateControlsInteraction()
  createBalls()
  startAnimation()
  isLoading.value = false
} catch (error) {
  console.error('初始化 3D 小球失败:', error)
  cleanupControls()
  cleanup()
```

```
    renderError.value = true
```

```
    isLoading.value = false
  }
}
function getLayoutConfig(count) {
  if (count >= 1000) {
    return { radius: 0.16, spacing: 0.4, layerGap: 0.64 }
  }
  if (count >= 500) {
    return { radius: 0.17, spacing: 0.42, layerGap: 0.68 }
  }
  if (count >= 100) {
    return { radius: 0.19, spacing: 0.46, layerGap: 0.72 }
  }
  if (count >= 50) {
    return { radius: 0.22, spacing: 0.56, layerGap: 0.72 }
  }
  return { radius: 0.26, spacing: 0.68, layerGap: 0.82 }
}
function createBallMaterial() {
  return new THREE.MeshStandardMaterial({
    color: 0xFFFFFF,
    roughness: 0.52,
    metalness: 0.02
  })
}
/**
 * Rebuilds the instanced mesh while keeping balls that were already visible
 * at their final scale. This avoids replaying the full entrance animation
 * whenever the explored number changes.
 *
 * @param {number} animationStartIndex – Index of the first newly added ball.
 */
function createBalls(animationStartIndex = 0) {
  if (!scene || !camera || !THREE) return
  if (instancedMesh) {
    scene.remove(instancedMesh)
    instancedMesh.geometry?.dispose()
    instancedMesh.material?.dispose()
    instancedMesh = null
  }
  const count = props.count
  const layoutConfig = getLayoutConfig(count)
  const positions = calculatePositions(count, layoutConfig)
  const bounds = getBounds(positions)
  const sphereGeometry = new THREE.SphereGeometry(layoutConfig.radius, 24, 24)
  instancedMesh = new THREE.InstancedMesh(sphereGeometry, createBallMaterial(), count)
  instancedMesh.castShadow = false
  instancedMesh.receiveShadow = false
  const dummy = new THREE.Object3D()
```

```
animationDummy = dummy
```

```
  const shouldAnimate = !prefersReducedMotion && animationStartIndex < count
  for (let i = 0; i < count; i++) {
    const pos = positions[i]
    const ballColor = getGroupedBallColor(count, i)
    dummy.position.set(pos.x, pos.y, pos.z)
    const scale = shouldAnimate && i >= animationStartIndex ? 0 : 1
    dummy.scale.set(scale, scale, scale)
    dummy.updateMatrix()
    instancedMesh.setMatrixAt(i, dummy.matrix)
    instancedMesh.setColorAt(i, ballColor)
  }
  instancedMesh.instanceMatrix.needsUpdate = true
  if (instancedMesh.instanceColor) {
    instancedMesh.instanceColor.needsUpdate = true
  }
  instancedMesh.userData = {
    positions: positions,
    animating: shouldAnimate,
    animationStartIndex
  }
  scene.add(instancedMesh)
  fitCameraToBounds(bounds)
  introStartTime = performance.now()
}
function fitCameraToBounds(bounds) {
  const center = bounds.center
  const layoutConfig = getLayoutConfig(props.count)
  const padding = layoutConfig.radius * 2.5 + 0.8
  const fovRad = camera.fov * (Math.PI / 180)
  const vFovHalf = fovRad / 2
  const aspect = camera.aspect || 1
  const hasDepth = bounds.depth > 0
  const depthWidth = hasDepth ? bounds.depth * 0.45 : 0
  const depthHeight = hasDepth ? bounds.depth * 0.16 : 0
  const distV = ((bounds.height + depthHeight + padding) / 2) / Math.tan(vFovHalf)
  const distH = ((bounds.width + depthWidth + padding) / 2) / Math.tan(vFovHalf) / aspect
  let baseDistance = Math.max(distV, distH) * 1.1 + (bounds.depth / 2)
  let distance = Math.max(baseDistance, 4.5)
  let heightBoost = Math.max(bounds.height * 0.08, 0.84)
  let lookTargetY = center.y
  if (props.count >= 100) {
    heightBoost = Math.max(bounds.height * 0.1, 1)
  }
  if (props.count >= 500) {
    heightBoost = Math.max(bounds.height * 0.12, 1.24)
    lookTargetY = center.y + bounds.height * 0.02
  }
  if (props.count >= 1000) {
    heightBoost = Math.max(bounds.height * 0.14, 1.42)
```

```
lookTargetY = center.y + bounds.height * 0.04
```

```
  }
  const cameraYaw = props.count >= 500 ? Math.PI / 7 : hasDepth ? Math.PI / 10 : 0
  const cameraX = center.x + Math.sin(cameraYaw) * distance
  const cameraZ = center.z + Math.cos(cameraYaw) * distance
  camera.position.set(cameraX, center.y + heightBoost, cameraZ)
  camera.lookAt(center.x, lookTargetY, center.z)
  if (controls) {
    controls.target.set(center.x, lookTargetY, center.z)
    controls.update()
  }
}
function calculatePositions(count, layoutConfig) {
  const positions = []
  const spacing = layoutConfig.spacing
  const rowSpacing = count > 10 ? spacing * 1.12 : spacing
  const layerGap = layoutConfig.layerGap
  const centeredOffset = 4.5
  for (let index = 0; index < count; index++) {
    const ones = index % 10
    const tens = Math.floor(index / 10) % 10
    const hundreds = Math.floor(index / 100)
    positions.push({
      x: (ones - centeredOffset) * spacing,
      y: (centeredOffset - tens) * rowSpacing,
      z: (hundreds - (Math.max(Math.ceil(count / 100), 1) - 1) / 2) * layerGap
    })
  }
  return positions
}
function getBounds(positions) {
  if (positions.length === 0) {
    return {
      width: 0,
      height: 0,
      depth: 0,
      center: { x: 0, y: 0, z: 0 }
    }
  }
  let minX = Infinity
  let maxX = -Infinity
  let minY = Infinity
  let maxY = -Infinity
  let minZ = Infinity
  let maxZ = -Infinity
  positions.forEach((p) => {
    minX = Math.min(minX, p.x)
    maxX = Math.max(maxX, p.x)
    minY = Math.min(minY, p.y)
    maxY = Math.max(maxY, p.y)
```

```
    minZ = Math.min(minZ, p.z)
```

```
    maxZ = Math.max(maxZ, p.z)
  })
  return {
    width: maxX - minX,
    height: maxY - minY,
    depth: maxZ - minZ,
    center: {
      x: (minX + maxX) / 2,
      y: (minY + maxY) / 2,
      z: (minZ + maxZ) / 2
    }
  }
}
function scheduleRender() {
  if (!animationId && !renderId && renderer && scene && camera) {
    renderId = requestAnimationFrame(renderOnce)
  }
}
function renderOnce() {
  renderId = null
  if (renderer && scene && camera) {
    controls?.update()
    renderer.render(scene, camera)
  }
}
function easeOutElastic(t) {
  const c4 = (2 * Math.PI) / 3
  return t === 0 ? 0 : t === 1 ? 1 : Math.pow(2, -10 * t) * Math.sin((t * 10 - 0.75) * c4) + 1
}
function animate(time) {
  if (!renderer || !scene || !camera || !instancedMesh?.userData.animating) {
    animationId = null
    return
  }
  controls?.update()
  const { positions, animationStartIndex } = instancedMesh.userData
  let allDone = true
  const elapsed = time - introStartTime
  for (let i = animationStartIndex; i < positions.length; i++) {
    const delay = (positions.length - 1 - i) * 1.5
    const ballElapsed = Math.max(0, elapsed - delay)
    const duration = 650
    let progress = ballElapsed / duration
    if (progress < 1) {
      allDone = false
    } else {
      progress = 1
    }
    const pos = positions[i]
```

```
    const scale = easeOutElastic(progress)
```

```
    animationDummy.position.set(pos.x, pos.y, pos.z)
    animationDummy.scale.set(scale, scale, scale)
    animationDummy.updateMatrix()
    instancedMesh.setMatrixAt(i, animationDummy.matrix)
  }
  instancedMesh.instanceMatrix.needsUpdate = true
  renderer.render(scene, camera)
  if (allDone) {
    instancedMesh.userData.animating = false
    animationId = null
    return
  }
  animationId = requestAnimationFrame(animate)
}
function startAnimation() {
  if (!instancedMesh?.userData.animating || animationId) {
    scheduleRender()
    return
  }
  introStartTime = performance.now()
  animationId = requestAnimationFrame(animate)
}
function cleanup() {
  if (animationId) {
    cancelAnimationFrame(animationId)
    animationId = null
  }
  if (renderId) {
    cancelAnimationFrame(renderId)
    renderId = null
  }
  if (renderer) {
    renderer.dispose()
    if (renderer.domElement.parentNode) {
      renderer.domElement.parentNode.removeChild(renderer.domElement)
    }
    renderer = null
  }
  if (instancedMesh) {
    instancedMesh.geometry?.dispose()
    instancedMesh.material?.dispose()
    instancedMesh = null
  }
  scene = null
  camera = null
  animationDummy = null
  isSceneReady.value = false
}
function syncRendererSize() {
```

```
if (!canvasRef.value || !renderer || !camera) return false
```

```
  const width = canvasRef.value.clientWidth
  const height = canvasRef.value.clientHeight
  if (!width || !height) return false
  camera.aspect = width / height
  camera.updateProjectionMatrix()
  renderer.setSize(width, height)
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
  return true
}
function handleResize() {
  if (!syncRendererSize()) return
  scheduleRender()
}
function toggleRotation() {
  isRotationEnabled.value = !isRotationEnabled.value
  updateControlsInteraction()
  scheduleRender()
}
function resetCameraView() {
  const positions = instancedMesh?.userData.positions
  if (!positions?.length) return
  fitCameraToBounds(getBounds(positions))
  scheduleRender()
}
async function retryRender() {
  isLoading.value = true
  renderError.value = false
  await nextTick()
  await initScene()
}
onMounted(() => {
  touchDevice.value = isTouchDevice()
  nextTick(() => {
    initScene()
    window.addEventListener('resize', handleResize)
    if (window.ResizeObserver && canvasRef.value) {
      resizeObserver = new ResizeObserver(() => {
        handleResize()
      })
      resizeObserver.observe(canvasRef.value)
    }
  })
})
onUnmounted(() => {
  cleanup()
  window.removeEventListener('resize', handleResize)
  resizeObserver?.disconnect()
})
watch(() => props.count, async (nextCount, previousCount) => {
```

```
isRotationEnabled.value = false
```

```
  if (renderError.value) return
  await nextTick()
  if (!scene || !renderer) return
  if (animationId) {
    cancelAnimationFrame(animationId)
    animationId = null
  }
  if (renderId) {
    cancelAnimationFrame(renderId)
    renderId = null
  }
  syncRendererSize()
  const animationStartIndex = nextCount > previousCount ? previousCount : nextCount
  createBalls(animationStartIndex)
  updateControlsInteraction()
  startAnimation()
})
</script>
<template>
  <div
    class="ball-array"
    :class="{
      'ball-array--compact': size === 'compact',
      'ball-array--has-controls': showRotationControls
    }"
  >
    <div v-if="isLoading" class="loading-indicator" aria-hidden="true">
      <span class="dot"></span>
      <span class="dot"></span>
      <span class="dot"></span>
    </div>
    <div
      v-if="renderError"
      class="render-fallback"
      role="group"
      :aria-label="ariaLabel"
      data-testid="ball-render-fallback"
    >
      <strong>暂时无法显示 3D 小球</strong>
      <span>可以重试显示，或刷新页面后再试。</span>
      <button
        class="render-retry-btn"
        type="button"
        data-testid="ball-render-retry"
        @click="retryRender"
      >
        <RotateCcw :size="16" aria-hidden="true" />
        <span>重试显示</span>
      </button>
```

```
</div>
```

```
    <div v-else ref="canvasRef" class="canvas-wrapper" role="img" :aria-label="ariaLabel"></div>
    <div v-if="showRotationControls" class="view-controls" aria-label="3D 视角控制">
      <button
        class="view-control-btn view-control-btn--toggle"
        :class="{ 'is-active': isRotationEnabled }"
        type="button"
        data-testid="rotation-toggle"
        :aria-label="isRotationEnabled ? '结束旋转并恢复页面滚动' : '启用小球旋转查看'"
        :aria-pressed="isRotationEnabled"
        @click="toggleRotation"
      >
        <Move3D :size="18" aria-hidden="true" />
        <span>{{ isRotationEnabled ? '完成' : '旋转查看' }}</span>
      </button>
      <button
        v-if="isRotationEnabled"
        class="view-control-btn view-control-btn--icon"
        type="button"
        data-testid="rotation-reset"
        aria-label="重置小球视角"
        @click="resetCameraView"
      >
        <RotateCcw :size="18" aria-hidden="true" />
      </button>
    </div>
  </div>
</template>
<style scoped>
.ball-array {
  position: relative;
  width: 100%;
  height: 100%;
  min-height: 236px;
  border-radius: var(--radius-lg);
  overflow: hidden;
  background: #f4f8ff;
  border: 1px solid rgba(146, 186, 236, 0.32);
  box-shadow: 0 12px 28px rgba(72, 116, 188, 0.1);
}
.canvas-wrapper {
  position: relative;
  width: 100%;
  height: 100%;
  min-height: 236px;
  border-radius: var(--radius-lg);
  overflow: hidden;
  background: transparent;
}
.canvas-wrapper canvas {
```

```
display: block;
```

```
  width: 100% !important;
  height: 100% !important;
}
.render-fallback {
  width: 100%;
  height: 100%;
  min-height: inherit;
  padding: 24px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 6px;
  color: var(--text-blue-light);
  text-align: center;
}
.render-fallback strong {
  color: var(--text-blue-dark);
  font-size: 15px;
}
.render-fallback span {
  font-size: 13px;
}
.render-retry-btn {
  min-height: 44px;
  margin-top: 8px;
  padding: 0 14px;
  border: 1px solid rgba(92, 157, 255, 0.18);
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.92);
  color: var(--brand-primary);
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  font-size: 13px;
  font-weight: 800;
  touch-action: manipulation;
  transition: background var(--duration-fast) var(--ease-out), border-color var(--duration-fast) var(--ease-out), transform var(--duration-
fast) var(--ease-out);
}
.render-retry-btn:active {
  transform: scale(0.96);
}
.render-retry-btn:focus-visible,
.view-control-btn:focus-visible {
  outline: none;
  box-shadow: 0 0 0 4px rgba(92, 157, 255, 0.18);
}
```

```
.ball-array--has-controls .canvas-wrapper {
  height: calc(100% - 56px);
```

```
  min-height: 0;
}
.view-controls {
  position: absolute;
  right: 8px;
  bottom: 6px;
  z-index: 2;
  min-height: 44px;
  display: flex;
  align-items: center;
  justify-content: flex-end;
  gap: 8px;
}
.view-control-btn {
  min-height: 44px;
  border: 1px solid rgba(92, 157, 255, 0.18);
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.94);
  color: var(--text-blue-dark);
  box-shadow: 0 6px 16px rgba(72, 116, 188, 0.1);
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  font-size: 13px;
  font-weight: 800;
  touch-action: manipulation;
  transition: background var(--duration-fast) var(--ease-out), border-color var(--duration-fast) var(--ease-out), color var(--duration-fast)
var(--ease-out);
}
.view-control-btn--toggle {
  padding: 0 13px;
}
.view-control-btn--icon {
  width: 44px;
  padding: 0;
}
.view-control-btn.is-active {
  border-color: rgba(74, 144, 226, 0.28);
  background: var(--brand-primary);
  color: #fff;
}
.ball-array--compact,
.ball-array--compact .canvas-wrapper {
  min-height: 138px;
  border-radius: var(--radius-md);
}
.loading-indicator {
  position: absolute;
```

```
  inset: 0;
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  gap: 8px;
  z-index: 1;
  pointer-events: none;
}
.loading-indicator .dot {
  width: 10px;
  height: 10px;
  border-radius: 50%;
  background: rgba(91, 143, 242, 0.6);
  animation: dot-pulse 1.2s ease-in-out infinite;
}
.loading-indicator .dot:nth-child(2) {
  animation-delay: 0.2s;
}
.loading-indicator .dot:nth-child(3) {
  animation-delay: 0.4s;
}
@keyframes dot-pulse {
  0%, 80%, 100% {
    transform: scale(0.6);
    opacity: 0.4;
  }
  40% {
    transform: scale(1);
    opacity: 1;
  }
}
@media (prefers-reduced-motion: reduce) {
  .loading-indicator .dot {
    animation: none;
    transform: scale(1);
    opacity: 0.6;
  }
}
@media (max-width: 420px) {
  .ball-array,
  .canvas-wrapper {
    min-height: 212px;
    border-radius: var(--radius-md);
  }
  .ball-array--compact,
  .ball-array--compact .canvas-wrapper {
    min-height: 124px;
  }
}
@media (max-width: 959px) and (max-height: 860px) {
  .ball-array,
```

```
.canvas-wrapper {
```

```
    min-height: 196px;
  }
}
</style>
// File: src/composables/useGame.js
import { ref, computed, toValue } from 'vue'
import { generateQuestions, checkAnswer, prepareQuestion } from '../utils/generator'
/**
 * 游戏核心逻辑 Composable
 * @param {Object|Ref<Object>} difficulty - 难度配置对象，支持响应式
 */
export function useGame(difficulty) {
  // 使用 toValue 获取实际值，支持响应式参数
  const difficultyValue = computed(() => toValue(difficulty))
  // 游戏状态
  const questions = ref([])
  const currentIndex = ref(0)
  const score = ref(0)
  const isComplete = ref(false)
  const correctCount = ref(0)
  const incorrectQuestions = ref([])  // 记录错题
  const startTime = ref(null)
  const endTime = ref(null)
  const currentQuestion = computed(() => questions.value[currentIndex.value])
  // 计算属性
  const progress = computed(() => {
    if (!questions.value.length) return 0
    return ((currentIndex.value) / questions.value.length) * 100
  })
  const accuracy = computed(() => {
    if (!questions.value.length) return 0
    const result = Math.round((correctCount.value / questions.value.length) * 100)
    return isNaN(result) ? 0 : result
  })
  const duration = computed(() => {
    if (!startTime.value) return 0
    const end = endTime.value || Date.now()
    const result = Math.floor((end - startTime.value) / 1000)
    return isNaN(result) ? 0 : result
  })
  const durationMs = computed(() => {
    if (!startTime.value) return 0
    const end = endTime.value || Date.now()
    const result = end - startTime.value
    return isNaN(result) ? 0 : Math.max(0, result)
  })
  /**
   * 开始游戏
   */
```

```
function startGame(options = {}) {
```

```
  const customQuestions = Array.isArray(options.questions) ? options.questions : null
  questions.value = customQuestions
    ? customQuestions.map((question, index) => prepareQuestion(question, index))
    : generateQuestions(difficultyValue.value)
  currentIndex.value = 0
  score.value = 0
  isComplete.value = false
  correctCount.value = 0
  incorrectQuestions.value = [] // 重置错题记录
  startTime.value = Date.now()
  endTime.value = null
}
/**
 * 提交答案
 * @param {number} answer - 用户输入的答案
 */
function submitAnswer(answer) {
  const question = currentQuestion.value
  if (!question) return
  const isCorrect = checkAnswer(question, answer)
  question.userAnswer = answer
  question.isCorrect = isCorrect
  if (isCorrect) {
    score.value += 10
    correctCount.value++
  } else {
    // 记录错题
    incorrectQuestions.value.push({
      id: question.id,
      operand1: question.operand1,
      operand2: question.operand2,
      operator: question.operator,
      result: question.result,
      missingPart: question.missingPart,
      userAnswer: answer,
      correctAnswer: question.answer,
      answer: question.answer
    })
  }
  // 不在这里切换到下一题，由外部控制
  return isCorrect
}
/**
 * 切换到下一题
 */
function nextQuestion() {
  if (currentIndex.value >= questions.value.length - 1) {
    completeGame()
  } else {
```

```
currentIndex.value++
```

```
    }
  }
  /**
   * 完成游戏
   */
  function completeGame() {
    endTime.value = Date.now()
    isComplete.value = true
  }
  /**
   * 获取游戏结果
   */
  function getResult() {
    return {
      score: score.value,
      correctCount: correctCount.value,
      totalCount: questions.value.length,
      accuracy: accuracy.value,
      duration: duration.value,
      durationMs: durationMs.value,
      difficulty: difficultyValue.value,
      completedAt: new Date().toISOString(),
      incorrectQuestions: incorrectQuestions.value  // 返回错题列表
    }
  }
  return {
    // 状态
    questions,
    currentIndex,
    score,
    isComplete,
    correctCount,
    incorrectQuestions,
    startTime,
    endTime,
    // 计算属性
    currentQuestion,
    progress,
    accuracy,
    duration,
    durationMs,
    // 方法
    startGame,
    submitAnswer,
    nextQuestion,
    completeGame,
    getResult
  }
}
```